

Instructions & Experimental Protocol

Vision-Based Human-Robot Handover on Quanser QArm

1 System Overview

This document details the operational configuration required to reproduce the Vision-Guided Human-Robot Handover system. The architecture splits processing into a high-speed perception layer executing in **Python 3.12.X** (YOLOv8 tracking + MediaPipe gestures) and a deterministic control loop running inside a finite state machine in **MATLAB/Simulink 2025a**, communicating locally via a binary UDP socket loop over Port 5005.

2 Prerequisites & Operational Conditions

2.1 Hardware Infrastructure

- **Robotic Manipulator:** Quanser QArm 4-DOF platform (powered, with external E-STOP within reach).
- **Sensing Subsystem:** Intel RealSense D415 RGB-D camera connected over a high-bandwidth USB 3.0 Type-C interface.
- **Compute Station:** Host workstation running concurrent Python and MATLAB runtime environments.

2.2 Software Requirements

- **Perception Layer:** Python 3.12.X setup within a virtual environment featuring `ultralytics`, `pyrealsense2`, `opencv-python`, `mediapipe`, and standard networking libraries.
- **Control Layer:** MATLAB / Simulink version **2025a** utilizing the DSP System Toolbox (for socket parsing) and the Quanser **QUARC** Real-Time Control target package.

3 Repository Architecture Overview

To successfully deploy the framework, ensure that your workspace matches the established repository structure:

- **MATLAB-Simulink Files/connecttoUDP_final.slx:** The primary Simulink model implementing the Finite State Machine, 3D Kalman State Filter, axis transformation matrix, and analytical Inverse Kinematics.
- **Python/vision_raw_udp_HD_bottle.py:** The production vision node configured for the reflective Volvic water bottle pick-and-place tracking scenario.
- **Python/vision_raw_udp_HD_ball.py:** The production vision node optimized for the tennis ball tracking scenario.
- **Detection Models/:** Directory containing the target convolutional neural network tensor weights (`model_bottle.pt` and `model_ball.pt`).

- **Simulation/:** Folder containing offline trajectory evaluation tools, including the interactive 3D graphical applications (`app1_GUI_trajectoryplanning.mlapp` and `potentialfield.mlapp`) and standalone graphing diagnostics (`trajectories_comparison.m` and `different_grafes.m`).

4 Step-by-Step Reproduction Guide

4.1 Step 1: Physical System Initialization

1. Securely bolt the Quanser QArm to the stable laboratory workbench surface.
2. Clear the manipulator's maximum extension envelope ($> 40cm$) of all physical structural obstacles.
3. Adjust the workspace lighting environment to minimize severe specular reflections off reflective targets.
4. Connect the 12V/1.5A external power supply transformer and twist the physical emergency stop switch clockwise to unlock hardware actuation.

4.2 Step 2: Activating the Vision Layer (Python)

1. Launch your workstation terminal, navigate to your workspace clone directory, and enter the Python 3.12.X virtual environment.
2. **Scenario Deployment:** Launch the perception script corresponding directly to your active target object.
 - For bottle tracking tasks: `python Python/vision_raw_udp_HD_bottle.py`
 - For tennis ball tracking tasks: `python Python/vision_raw_udp_HD_ball.py`
3. Wait for the live OpenCV rendering feed window to populate on-screen. Check that the high-throughput pipeline is hitting its nominal 50–60 FPS specification profile and actively streaming target bounding data vectors to localhost.

4.3 Step 3: Compiling the Control Layer (MATLAB/Simulink)

1. Launch MATLAB 2025a and open the control block diagram:
`MATLAB-Simulink Files/connecttoUDP_final.slx`.
2. Verify that the HIL Initialize block parameters match your active local workstation hardware port interface assignments.
3. Ensure that the system parameters match the physical constraints (e.g., confirming the spatial commit zone threshold matches the structural target volume boundary of $\leq 18cm$).

4.4 Step 4: Live Hardware-in-the-Loop Execution

1. In the Simulink menu banner workspace, select the **Hardware** configuration tab.
2. Click on the **Monitor/Tune** execution block (do not click the standard simulation "Run" button, as the control loop must compile via QUARC Coder to map outputs straight onto real-world motor registers).
3. The manipulator will build the model blocks and enter its autonomous state machine behavior loop, moving to home and sweeping the horizontal space via a sinusoidal hunting trajectory.